Análise e Projeto do Google Congestion Control para Comunicação em Tempo Real na Web (WebRTC)

Gaetano Carlucci (Politecnico di Bari, Itália) · Luca De Cicco (Télécom SudParis, França) · Stefan Holmer (Google, Suécia) · Saverio Mascolo (Politecnico di Bari, Itália)

MMSys'16, 10–13 de maio de 2016, Klagenfurt, Áustria. DOI: 10.1145/2910017.2910605

Tradução não oficial para o português (pt-BR) de *Analysis and Design of the Google Congestion Control for Web Real-time Communication (WebRTC).* Termos técnicos consagrados (como *bitrate*, *throughput*, *jitter*) foram mantidos em inglês quando soam mais naturais assim. Figuras e imagens foram omitidas (permanecem apenas as legendas traduzidas); tabelas e equações foram reproduzidas. A lista de referências é mantida no idioma original. Em caso de dúvida, vale o texto original.

RESUMO

Aplicações de videoconferência exigem baixa latência e alta largura de banda. O TCP padrão não é adequado para videoconferência, pois seus mecanismos de confiabilidade e de entrega em ordem induzem grande latência. Recentemente, a ideia de usar o gradiente de atraso para inferir congestionamento voltou a aparecer e vem ganhando força. Neste artigo apresentamos um algoritmo baseado na estimativa, por meio de um filtro de Kalman, da variação do atraso unidirecional fim a fim sofrida pelos pacotes que viajam de um remetente até um destino. Essa estimativa é comparada a um limiar adaptativo para limitar dinamicamente a taxa de envio. O algoritmo de controle foi implementado sobre o protocolo RTP/RTCP e é usado atualmente no Google Hangouts e na pilha WebRTC do Chrome. Foram realizados experimentos para avaliar o desempenho do algoritmo com capacidade de enlace variável, na presença de tráfego concorrente heterogêneo ou homogêneo e de tráfego no caminho de retorno.

Palavras-chave: Comunicação em tempo real, controle de congestionamento, WebRTC

1 INTRODUÇÃO

A QoE (qualidade de experiência) em videoconferência é afetada não apenas pelo goodput, geralmente relacionado à qualidade da imagem do vídeo, mas também pela latência da conexão, que deve ser mantida a mais baixa possível para permitir uma comunicação fluida [4]. É por isso que os fluxos gerados por essas aplicações são considerados sensíveis a atraso. É sabido que o TCP, que ainda é o protocolo de transporte mais usado na Internet, não é adequado para entregar o tráfego gerado por aplicações de tempo real. O principal motivo é que o TCP privilegia a confiabilidade e a entrega em ordem, por meio de retransmissões, em detrimento da entrega dos pacotes em tempo hábil. Embora alguns trabalhos tenham investigado as implicações do uso do TCP para tráfego VoIP [5, 29], o caso da videoconferência — que envolve taxas de dados muito maiores — nunca foi estudado. Assim, as aplicações de videoconferência têm usado tradicionalmente o UDP, deixando o controle de congestionamento para a camada de aplicação. Já foi demonstrado que diversas aplicações conhecidas de videoconferência se adaptam, ao menos em certa medida, à largura de banda disponível na rede, como no caso do Skype [8] e de outras aplicações [33].

Esta área de pesquisa vem recebendo atenção renovada graças à iniciativa WebRTC, que visa padronizar um framework interoperável e eficiente para comunicação em tempo real em navegadores Web usando RTP [1]. Neste artigo apresentamos o Google Congestion Control (GCC) [15], um algoritmo compatível com o framework WebRTC, implementado nos navegadores Chrome e usado no Google Hangouts. O algoritmo foi projetado para funcionar com os protocolos RTP/RTCP e se baseia na ideia de usar o gradiente de atraso para inferir congestionamento. Para isso, um filtro de Kalman é projetado para produzir uma estimativa do gradiente do atraso de fila, que é comparada a um limiar adaptativo a fim de detectar congestionamento.

O restante do artigo está organizado da seguinte forma: a Seção 2 revisa a literatura relevante sobre controle de congestionamento para fluxos sensíveis a atraso; a Seção 3 descreve o algoritmo proposto; a Seção 4 justifica as escolhas de projeto empregadas para detectar o congestionamento da rede; a Seção 5 apresenta o testbed experimental e as métricas utilizadas; a Seção 6 mostra os resultados experimentais; e, por fim, a Seção 7 conclui o artigo.

2 TRABALHOS RELACIONADOS

O TCP tradicional, baseado em perdas, não é adequado para tráfego de videoconferência, pois seu controle de congestionamento sonda continuamente a largura de banda disponível da rede, introduzindo ciclos periódicos nos quais as filas da rede são primeiro preenchidas e depois esvaziadas. Essas oscilações de fila induzem um componente de atraso estocástico variável no tempo, que se soma ao tempo de propagação e torna problemáticas as comunicações sensíveis a atraso. A ideia de que o atraso da rede está correlacionado ao congestionamento foi proposta já em 1989 [16]. No entanto, diversos problemas relacionados às medições de atraso em algoritmos baseados em atraso foram apontados [23], especialmente em ambientes sem fio [13] e quando o gargalo é compartilhado com fluxos baseados em perdas [12]. Esta seção revisa a literatura relevante sobre controle de congestionamento para fluxos sensíveis a atraso em redes de longa distância.

O uso do RTT para inferir congestionamento. Os primeiros esforços voltados à redução do atraso de fila surgiram no domínio de pesquisa de controle de congestionamento do TCP e, consequentemente, muitos algoritmos para tráfego de tempo real têm raízes nessa literatura. O primeiro algoritmo de controle de congestionamento projetado especificamente para conter a latência fim a fim por meio de medições de atraso é o trabalho seminal de Jain, de 1989 [16]. Desde então, várias variantes de controle de congestionamento do TCP baseadas em atraso foram propostas, como o TCP Vegas [3]. Tipicamente, algoritmos baseados em atraso exigem o conhecimento das estatísticas de RTT fim a fim para estabelecer limiares de atraso apropriados, usados para inferir congestionamento, como no caso do TCP

Vegas [3] e do TCP FAST [30]. Já foi demonstrado que, quando o RTT é usado como métrica de congestionamento, pode-se obter baixa utilização do canal na presença de tráfego reverso ou ao competir com fluxos baseados em perdas [12]. Vale mencionar que o problema do tráfego reverso é crucial no contexto da videoconferência, já que os fluxos de vídeo são enviados nos dois sentidos.

Projeto para RTP/RTCP. O foco deste artigo é o projeto de um algoritmo de controle de congestionamento para RTP/RTCP sobre UDP. Embora essa área de pesquisa tenha sido ativa no passado [27], hoje ela volta a atrair atenção graças à iniciativa conjunta WebRTC do W3C e do IETF, que visa viabilizar a comunicação em tempo real interoperável entre navegadores Web [1]. Três algoritmos fim a fim foram propostos no grupo de trabalho RMCAT do IETF: 1) o Network Assisted Dynamic Adaptation (NADA) [35], um algoritmo baseado em perdas/atraso que depende de medições de atraso unidirecional; 2) o Self-Clocked Rate Adaptation for Multimedia (SCREAM) [17], que herda algumas ideias do LEDBAT; 3) o Google Congestion Control (GCC) [15], que será descrito a seguir.

O uso do atraso unidirecional para inferir congestionamento. Outra classe de algoritmos defende o uso de medições de atraso unidirecional para eliminar a sensibilidade à congestão do caminho reverso. Exemplos são o LEDBAT (sobre UDP) [26] e o TCP-LP [18]. Em particular, o LEDBAT [26] aumenta sua janela de congestionamento a uma taxa proporcional à distância entre o atraso unidirecional medido e um atraso-alvo fixo τ. Foi demonstrado que o LEDBAT sofre do chamado "efeito latecomer" (retardatário): quando dois fluxos compartilham o mesmo gargalo, o segundo fluxo tipicamente faz o primeiro passar fome (starvation) [7].

O uso do gradiente de atraso para inferir congestionamento. A ideia de empregar o gradiente do RTT para inferir congestionamento foi usada recentemente para superar o referido "efeito latecomer". Alguns exemplos são o CDG [14] e o Verus [34]. O CDG [14] foi projetado com o objetivo de coexistir com fluxos baseados em perdas mantendo baixo o atraso fim a fim. O Verus [34] foi projetado especificamente para redes celulares, nas quais variações súbitas da capacidade do enlace tornam desafiador o projeto do controle de congestionamento. Recentemente, mostrou-se que medições precisas do gradiente de atraso são viáveis em redes de data center por meio de timestamps de hardware das NICs [19].

Outras abordagens. Entre os algoritmos de controle de congestionamento propostos recentemente que não inferem congestionamento explicitamente a partir de medições de atraso de rede, citamos o Sprout [32], o Remy [31] e o FBRA [20]. O Sprout [32] adota uma abordagem estocástica para conter atrasos enquanto maximiza o throughput; o artigo mostra que o algoritmo supera aplicações como Skype, Facetime e Google Hangouts em um cenário de fluxo único; contudo, o desempenho com múltiplos fluxos compartilhando um gargalo ainda não foi avaliado. O Remy [31] é um framework para gerar algoritmos de controle de congestionamento. Ao definir uma função de utilidade baseada nos requisitos dos usuários, o Remy emprega o conhecimento a priori da rede para treinar uma máquina a aprender esquemas de controle de congestionamento. O FBRA [20] propõe um algoritmo de controle de congestionamento baseado em FEC: a ideia é sondar a largura de banda disponível por meio de pacotes FEC e, em caso de perdas devidas a congestionamento, os pacotes redundantes ajudam a recuperar os pacotes perdidos.

3 FUNDAMENTAÇÃO

Esta seção fornece informações de base sobre o algoritmo apresentado no artigo. Começamos definindo o gradiente do atraso de fila e, em seguida, mostramos como ele é empregado no algoritmo proposto para detectar congestionamento.

3.1 Gradiente do atraso de fila

O gradiente do atraso de fila (unidirecional) é definido como a derivada do atraso de fila $T_q(t)$, que pode ser modelado como $T_q(t) = q(t)/C$, em que C é a capacidade do enlace gargalo e $q(t)$ é o comprimento da fila medido em bits. Assim, o gradiente do atraso de fila $\dot{T}_q(t)$ é igual a:

$$\dot{T}_q(t) = \dot{q}\frac{t}{C} \tag{1}$$

A derivada do comprimento da fila, $\dot{q}(t)$, pode ser modelada da seguinte forma:

$$\dot{q}(t) = \begin{cases} r(t) - C & 0 \leq q(t) \leq q_M \\ 0 & \text{caso contrário} \end{cases} \tag{2}$$

em que $r(t)$ é a taxa de preenchimento da fila medida em bits por segundo e q_M é o comprimento máximo da fila. Por definição, $\dot{q}(t)$ é igual a zero e, portanto, $\dot{T}_q(t) = 0$, quando o comprimento da fila $q(t)$ permanece constante. Isso pode ocorrer em três condições diferentes: 1) quando a fila é zero devido à subutilização do canal, isto é, quando a taxa de preenchimento $r(t)$ está abaixo da capacidade do enlace C e a fila acaba se esvaziando; 2) quando a fila está cheia devido a congestionamento persistente, isto é, quando a taxa de preenchimento $r(t)$ excede a capacidade do enlace C; 3) quando a taxa de entrada $r(t)$ coincide exatamente com C. No terceiro caso, isto é, $r(t) = C$, a fila permanece constante em um valor $q \in [0, q_M]$. Tal situação, definida em [22] como *standing queue* (fila permanente), é considerada indesejável, pois atrasa continuamente o tráfego que chega. O algoritmo proposto visa manter a fila a menor possível sem subutilizar o enlace. Para atingir esse objetivo, o algoritmo precisa sondar a largura de banda disponível, aumentando sua taxa de envio até que uma variação positiva do atraso de fila seja detectada. Nesse ponto, a taxa de envio é reduzida. Por esse motivo, argumentamos que algum atraso de fila precisa ser induzido para que o controle de congestionamento baseado em atraso funcione.

3.2 Arquitetura do algoritmo

A Figura 1 mostra a arquitetura geral de controle empregada pelo algoritmo Google Congestion Control (GCC). O remetente emprega um socket UDP para enviar pacotes RTP e receber relatórios de feedback RTCP do receptor. O algoritmo possui dois componentes: 1) um controlador baseado em atraso, situado no receptor, que calcula uma taxa A_r que é devolvida ao remetente com o objetivo de conter o atraso; 2) um controlador baseado em perdas, situado no remetente, que calcula a taxa de envio alvo A_s, a qual não pode exceder A_r.

Figura 1: Arquitetura do Google Congestion Control. O remetente (encoder, pacer, padder/FEC, controlador baseado em perdas) define $A = \min(A_r, A_s)$; o receptor (filtro de chegada, detector de sobreuso, limiar adaptativo, controlador de taxa remoto, processamento de REMB) devolve A_r via RTCP/REMB.
(imagem omitida nesta tradução; consulte o artigo original)

Figura 2: Medição do gradiente de atraso unidirecional.
(imagem omitida nesta tradução; consulte o artigo original)

A seguir, apresentamos uma descrição do algoritmo. Detalhes técnicos também podem ser encontrados no draft do IETF [15] e no repositório de código do WebRTC[1] do navegador Chromium.

3.3 O controlador baseado em atraso

O controlador do lado do receptor é um algoritmo de controle de congestionamento baseado em atraso que calcula A_r de acordo com a seguinte equação:

$$A_r(t_i) = \begin{cases} \eta A_r(t_{i-1}) & \sigma = \text{Aumento} \\ \alpha R_r(t_i) & \sigma = \text{Redução} \\ A_r(t_{i-1}) & \sigma = \text{Manutenção} \end{cases} \tag{3}$$

em que t_i denota o instante em que o i-ésimo quadro de vídeo é recebido, $\eta = 1,05$, $\alpha = 0,85$ e $R_r(t_i)$ é a taxa de recepção medida nos últimos 500 ms. A Figura 1 mostra um diagrama de blocos do controlador baseado em atraso, composto por vários componentes descritos a seguir.

O *controlador de taxa remoto* é uma máquina de estados finitos (ver Figura 4) em que o estado σ de (3) é alterado pelo sinal s produzido pelo detector de sobreuso com base na saída $m(t_i)$ do filtro de tempo de chegada. O bloco de *limiar adaptativo* define dinamicamente o limiar $\gamma(t_i)$ usado pelo detector de sobreuso. O *processamento de REMB* decide quando enviar uma mensagem REMB[2] com base no valor da taxa A_r. Por fim, é importante notar que $A_r(t_i)$ é limitada superiormente por $1,5R_r(t_i)$.

A seguir, apresentamos uma descrição detalhada de cada bloco.

Figura 3: Sinalização do detector de sobreuso (sinais *overuse*, *normal* e *underuse* conforme $m(t_i)$ cruza $\pm\gamma(t_i)$).
(imagem omitida nesta tradução; consulte o artigo original)

O filtro de tempo de chegada. O objetivo deste bloco é produzir uma estimativa $m(t_i)$ do gradiente do atraso unidirecional. Para isso, empregamos um filtro de Kalman que estima $m(t_i)$ com base no gradiente de atraso unidirecional medido $d_m(t_i)$, calculado da seguinte forma (ver Figura 2):

$$d_m(t_i) = (t_i - t_{i-1}) - (T_i - T_{i-1}) \tag{4}$$

em que T_i é o instante em que o primeiro pacote do i-ésimo quadro de vídeo foi enviado e t_i é o instante em que o último pacote que compõe o quadro de vídeo foi recebido.

O detector de sobreuso. A cada instante t_i em que um quadro de vídeo é recebido, o detector de sobreuso produz um sinal s que conduz o estado σ da máquina de estados (3) com base em $m(t_i)$ e em um limiar $\gamma(t_i)$. A Figura 3 mostra como s é gerado: quando $m(t_i) > \gamma(t_i)$, o algoritmo começa a medir o tempo passado nessa condição e, se ele for maior que 100 ms, o sinal de sobreuso (*overuse*) é gerado. Por outro lado, se $m(t_i)$ cair abaixo de $-\gamma(t_i)$, é gerado o sinal de subuso (*underuse*), enquanto o sinal normal é disparado quando $-\gamma(t_i) \leq m(t_i) \leq \gamma(t_i)$.

Controlador de taxa remoto. Este bloco calcula A_r de acordo com (3), usando o sinal s produzido pelo detector de sobreuso, que conduz a máquina de estados finitos mostrada na Figura 4. O objetivo da máquina de estados finitos é minimizar os atrasos de fila nos buffers ao longo do caminho fim a fim. A lógica é a seguinte: quando os buffers do gargalo começam a se acumular, o gradiente de atraso unidirecional estimado $m(t_i)$ torna-se positivo. O detector de sobreuso detecta essa variação e dispara um sinal de sobreuso, que leva a máquina ao estado de "redução". Como resultado, a taxa de envio é reduzida e o buffer do gargalo começa a ser drenado, até o ponto em que o gradiente de atraso unidirecional estimado $m(t_i)$ se torna negativo. Um sinal de subuso é então disparado, o que leva a máquina ao estado de "manutenção". A máquina permanece no estado de "manutenção" até que o buffer do gargalo seja esvaziado. Quando isso ocorre, $m(t_i)$ se aproxima de 0 e o detector de sobreuso gera um sinal normal, que leva a máquina ao estado de "aumento".

Figura 4: Máquina de estados finitos do controlador de taxa remoto. Estados: aumento ($A_r(t_i) = \eta A_r(t_{i-1})$), redução ($A_r(t_i) = \alpha R(t_i)$) e manutenção ($A_r(t_i) = A_r(t_{i-1})$), com transições disparadas pelos sinais *normal*, *overuse* e *underuse*.
(imagem omitida nesta tradução; consulte o artigo original)

Limiar adaptativo. O objetivo deste bloco, mostrado na Figura 1, é adaptar a sensibilidade do algoritmo às variações de atraso conforme as condições da rede. Em particular, este bloco evita dois problemas: 1) a inibição do controlador baseado em atraso quando o tamanho da fila do gargalo ao longo do caminho não é suficientemente grande e 2) a starvation (inanição) dos fluxos GCC na presença de tráfego TCP concorrente baseado em perdas. A entrada deste bloco é o gradiente de atraso estimado $m(t_i)$; a saída é o valor dinâmico do limiar $\gamma(t_i)$, que é fornecido ao detector de sobreuso. Mais detalhes sobre este bloco são apresentados na Seção 4.2.

Processamento de REMB. Este bloco notifica o remetente da taxa calculada A_r por meio de mensagens REMB. As mensagens REMB são enviadas a cada 1 s ou imediatamente, se $A_r(t_i) < 0,97A_r(t_{i-1})$, isto é, quando A_r diminuiu mais de 3%.

3.4 O controlador baseado em perdas

O controlador baseado em perdas complementa o controlador baseado em atraso nos casos em que há perdas medidas. O algoritmo atua toda vez que, no instante t_k, o remetente recebe a k-ésima mensagem de relatório RTCP ou uma mensagem REMB que carrega A_r. O formato REMB é uma extensão do protocolo RTCP que está em discussão no IETF. Os relatórios RTCP incluem a fração de pacotes perdidos $f_l(t_k)$, calculada conforme descrito na RFC do RTP. O remetente usa $f_l(t_k)$ para calcular a taxa de envio $A_s(t_k)$ de acordo com a seguinte equação:

$$A_s(t_k) = \begin{cases} A_s(t_{k-1})(1 - 0,5f_l(t_k)) & f_l(t_k) > 0 \\ 1 & \\ 1 & \\ 05A_s(t_{k-1}) & f_l(t_k) < 0 \\ 02 & \\ A_s(t_{k-1}) & \text{caso contrário} \end{cases} \tag{5}$$

A lógica de (5) é simples: 1) quando a fração de perdas é considerada pequena ($0,02 \leq f_l(t_k) \leq 0,1$), A_s é mantida constante; 2) se uma fração de perdas alta é estimada ($f_l(t_k) > 0,1$), a taxa é reduzida multiplicativamente; e 3) quando a fração de perdas é considerada desprezível ($f_l(t_k) < 0,02$), a taxa é aumentada multiplicativamente.

[1] https://chromium.googlesource.com/external/webrtc/+/master/webrtc/
[2] http://tools.ietf.org/html/draft-alvestrand-rmcat-remb-03

3.5 Atuação da taxa de envio

Nesta seção descrevemos o mecanismo empregado para atuar sobre a taxa de envio calculada pelos dois controladores. A Figura 1 mostra que o remetente define a taxa de bits alvo A igual ao mínimo entre A_r e A_s. A taxa alvo A é fornecida tanto ao Pacer quanto ao Encoder. O Encoder se esforça para produzir uma taxa de bits o mais próxima possível desse alvo. Contudo, o encoder não consegue alterar a taxa com a mesma frequência que a taxa do Pacer, para evitar a oscilação (*flickering*) da qualidade do vídeo, sabidamente prejudicial à QoE [21]. Por esse motivo, o encoder pode não ser capaz de produzir uma taxa que corresponda exatamente ao alvo A. Caso o encoder produza uma taxa maior que o alvo A, o Pacer pode drenar sua fila a uma taxa $f \cdot A$, em que f é o fator de pacing, igual a 1,5. Dessa forma, o pacer consegue esvaziar rapidamente sua fila, evitando atrasos de fila no remetente. Por outro lado, sempre que o Pacer produzir uma taxa menor que o alvo A, podem ser adicionados padding ou FEC. Como resultado desse mecanismo, produz-se uma taxa de bits média de envio igual a A. A Figura 5 mostra o caso de um fluxo GCC sobre um enlace de 1 Mbps. A figura mostra que a taxa real gerada pelo mecanismo de envio do GCC acompanha bem a taxa de bits alvo A definida pelo algoritmo de controle de congestionamento.

Figura 5: Atuação da taxa de envio (taxa GCC A e capacidade do enlace, em kbps, ao longo de 300 s).
(imagem omitida nesta tradução; consulte o artigo original)

4 DETECÇÃO DE CONGESTIONAMENTO

Nesta seção, justificamos as escolhas de projeto adotadas para inferir o congestionamento da rede. Em particular, tratamos de dois aspectos cruciais: 1) a forma como a estimativa $m(t_i)$ é produzida por meio de um filtro de Kalman e 2) o projeto do limiar adaptativo usado para detectar congestionamento.

4.1 Filtro de Kalman

4.1.1 Projeto

Assumimos que o gradiente de atraso é afetado por um ruído de jitter. Para filtrar esse ruído e produzir a estimativa $m(t)$, emprega-se um filtro de Kalman. O projeto do filtro de Kalman baseia-se em um modelo de sistema composto por: 1) uma equação dinâmica de estado, que descreve a dinâmica do vetor de estado, e 2) uma equação estática de saída, que descreve a relação entre o vetor de estado e a medição [11].

Consideremos o seguinte modelo do vetor de estado[3], composto por dois componentes:

$$\boldsymbol{\theta}(t_i) = \begin{pmatrix} 1/C(t_i) \\ \mu(t_i) \end{pmatrix} \tag{6}$$

em que $1/C(t_i)$ é o inverso da capacidade do enlace gargalo e $\mu(t)$ é o gradiente do atraso de fila unidirecional; t_i denota o instante em que o i-ésimo quadro de vídeo é recebido. O modelo do sistema é dado por:

$$\begin{cases} \boldsymbol{\theta}(t_{i+1}) = \boldsymbol{\theta}(t_i) + \boldsymbol{w}(t_i) & \text{(equação de estado)} \\ d_m(t_i) = \frac{\Delta L(t_i)}{C(t_i)} + \mu(t_i) + n(t_i) & \text{(equação de saída)} \end{cases} \tag{7}$$

[3]Ao longo do artigo, vetores são representados em negrito para diferenciá-los de variáveis escalares.

A evolução do estado no instante t_{i+1} é dada pelo seu estado no instante t_i somado a um ruído de estado gaussiano $\boldsymbol{w}(t_i)$. O modelo da variação de atraso unidirecional medida $d_m(t_i)$ é dado pela soma de três componentes: 1) a variação do tempo de transmissão, dada pela razão entre a variação de tamanho de dois quadros de vídeo consecutivos $\Delta L(t_i)$ e a capacidade do enlace gargalo $C(t_i)$; 2) o *gradiente do atraso de fila* $\mu(t_i)$; e 3) $n(t_i)$, modelado como ruído gaussiano. No entanto, mostramos a seguir que o sistema descrito por (7) pode não ter a propriedade de observabilidade de sistemas dinâmicos [11]. Um sistema dinâmico é observável se existe um N finito tal que, para todo estado inicial $\boldsymbol{\theta}(t_0)$, N medições $\{d(t_0), d(t_1), ..., d(t_{N-1})\}$ distinguem de forma única o estado inicial $\boldsymbol{\theta}(t_0)$ [11]. Podemos provar a seguinte proposição:

Proposição 1. *O sistema descrito pela eq. (7), assumido livre de ruído, não é observável se a variação de tamanho $\Delta L(t_i)$ de duas amostras consecutivas for constante.*

Prova. Consideramos duas medições consecutivas, $N = 2$, já que temos um sistema bidimensional:

$$\begin{cases} d(t_0) = \frac{\Delta L(t_0)}{C(t_0)} + \mu(t_0) \\ d(t_1) = \frac{\Delta L(t_1)}{C(t_1)} + \mu(t_1) = \frac{\Delta L(t_1)}{C(t_0)} + \mu(t_0) \end{cases} \tag{8}$$

De (7) temos que $\boldsymbol{\theta}(t_1) = \boldsymbol{\theta}(t_0)$ em um sistema livre de ruído. A solução $\boldsymbol{\theta}(t_0)$ de (8) é única se e somente se as duas equações forem linearmente independentes ou, em outras palavras, se a matriz de observabilidade:

$$\mathcal{O} = \begin{pmatrix} \Delta L(t_0) & 1 \\ \Delta L(t_1) & 1 \end{pmatrix} \tag{9}$$

tiver posto $\text{rank}(\mathcal{O})$ igual a 2 [11]. Essa condição não se verifica se $\Delta L(t_0) = \Delta L(t_1)$, o que conclui a prova. □

Assim, de acordo com a Proposição 1, o sistema é observável se e somente se $\Delta L(t_i)$ mudar a cada passo. Infelizmente, essa condição não se verifica em geral e, nesses casos, a estimação do estado $\boldsymbol{\theta}(t_i)$ é indeterminada.

Por isso propomos simplificar o sistema da seguinte forma:

$$\begin{cases} \overline{m}(t_{i+1}) = \overline{m}(t_i) + w(t_i) & \text{(equação de estado)} \\ d_m(t_i) = \overline{m}(t_i) + n(t_i) & \text{(equação de saída)} \end{cases} \tag{10}$$

em que $\overline{m}(t)$ é agora a única variável de estado, que é o modelo do gradiente de atraso unidirecional. Essa escolha também pode ser relacionada ao fato de que conseguimos medir o gradiente de atraso unidirecional como um todo, mas não conseguimos distinguir a parte devida ao enfileiramento (isto é, $\mu(t_i)$) da parte devida ao tempo de transmissão e de processamento. Além disso, argumentamos que a contribuição dada pela variação do tempo de transmissão $\Delta L(t_i)/C(t_i)$ pode ser considerada desprezível em relação ao gradiente do atraso de fila $\mu(t_i)$. Agora, como o sistema (10) é escalar, a observabilidade é sempre garantida.

O ruído de estado $w(t_i)$ é modelado como um processo gaussiano estacionário de média zero e variância $Q(t_i) = E\left[w(t_i)^2\right]$. De modo análogo a $w(t_i)$, o ruído de medição $n(t_i)$ — que leva em conta o jitter da rede — é considerado um processo gaussiano estacionário de média zero e variância $\sigma_n^2(t_i) = E\left[n(t_i)^2\right]$.

Tanto a variância do ruído de estado quanto a do ruído de medição são parâmetros importantes, que precisam ser ajustados adequadamente. Essa questão será tratada na Seção 4.1.2.

A Figura 6 mostra como uma nova estimativa de estado $m(t_i)$ é obtida: a cada passo, a *inovação* ou *resíduo* $z(t_i)$ é multiplicada pelo ganho de Kalman $K(t_i)$, que fornece a correção da estimativa $m(t_i)$ de acordo com:

Figura 6: Estimação de estado com um filtro de Kalman.
(imagem omitida nesta tradução; consulte o artigo original)

$$m(t_i) = (1 - K(t_i)) \cdot m(t_{i-1}) + K(t_i) \cdot (d_m(t_i)) \tag{11}$$

A Eq. (11) mostra que o filtro de Kalman, neste caso específico, é equivalente a um filtro EWMA composto por dois termos aditivos [9]: o primeiro leva em conta a contribuição da estimativa de estado anterior $m(t_{i-1})$ e o segundo a contribuição da medição $d_m(t_i)$. O ganho $K(t_i)$ equilibra essas duas contribuições: se $K(t_i)$ é grande, dá-se mais peso à medição; inversamente, quando $K(t_i)$ é pequeno, dá-se mais peso à estimativa de estado. O ganho de Kalman é atualizado de acordo com as variâncias do ruído de processo e do ruído de medição:

$$K(t_i) = \frac{P(t_{i-1}) + Q(t_i)}{(P(t_{i-1}) + Q(t_i)) + \sigma_n^2(t_i)} \tag{12}$$

em que $P(t_k)$ é a variância do erro do sistema, definida como:

$$P(t_i) = E\left[(\overline{m}(t_i) - m(t_i))^2\right] \tag{13}$$

que queremos minimizar em regime estacionário. Esse valor pode ser calculado recursivamente a cada passo da seguinte forma:

$$P(t_i) = (1 - K(t_i)) \cdot (P(t_{i-1}) + Q(t_i)) \tag{14}$$

Uma interpretação de (12) é a seguinte: à medida que a variância do erro de medição $\sigma_n^2(t_i)$ se aproxima de zero, confia-se mais na medição e, portanto, o ganho de Kalman aumenta. Vice-versa, à medida que a covariância do erro P se aproxima de zero, confia-se menos na medição e o ganho de Kalman diminui.

4.1.2 Implementação

Vários parâmetros precisam ser ajustados para implementar o filtro de Kalman: 1) as estatísticas dos processos de ruído de estado e de medição e 2) as condições iniciais de P.

Começamos considerando a variância do ruído de estado Q. Primeiro, assumimos que Q é constante e realizamos um conjunto de experimentos para ajustá-la adequadamente.

A Figura 7 mostra o efeito de Q tanto sobre a estimativa do gradiente de atraso unidirecional $m(t)$ quanto sobre a taxa de envio. Quatro valores diferentes de Q foram testados no caso de um único fluxo GCC sobre um enlace de 1 Mbps. A Figura 7 também apresenta a dinâmica do RTT, que é correlacionada ao gradiente de atraso unidirecional. Notamos que, quando $Q = 10^{-4}$, a reação às variações de RTT é lenta, provocando grandes atrasos de fila, claramente visíveis na dinâmica correspondente do RTT (Figura 7(d)). A Figura 7(c) mostra que, quando $Q = 10^{-3}$, o gradiente de atraso estimado está bem correlacionado com a dinâmica do RTT: em correspondência a aumentos do RTT, $m(t)$ exibe picos positivos e, em correspondência a reduções do RTT, $m(t)$ exibe picos negativos. Por fim, $m(t)$ é conduzido a 0 quando o RTT é mantido próximo ao atraso de propagação $\mathrm{RTT}_{\min} = 50$ ms, já que nesse caso o gradiente de atraso é zero. Quando se usam $Q = 10^{-1}$ e $Q = 10^{-2}$, $m(t)$ torna-se muito sensível ao ruído de jitter, causando subutilização do enlace, pois a reação ao congestionamento pode ser conduzida pelo ruído de jitter. Com base nessa análise, definimos Q igual a:

Figura 7: Efeito da variância do ruído de estado Q sobre o gradiente medido, o RTT e a taxa de envio. (a) $Q = 10^{-1}$; (b) $Q = 10^{-2}$; (c) $Q = 10^{-3}$; (d) $Q = 10^{-4}$.
(imagem omitida nesta tradução; consulte o artigo original)

$$Q = 10^{-3} \tag{15}$$

Concentramo-nos agora na questão de estimar a variância do ruído de medição $\hat{\sigma}_n^2(t_i)$. Estimamos $\hat{\sigma}_n^2(t_i)$ usando um filtro de média móvel exponencial do *resíduo* $z(t_i) = d_m(t_i) - m(t_{i-1})$:

$$\hat{\sigma}_n^2(t_i) = \beta \cdot \hat{\sigma}_n^2(t_{i-1}) + (1 - \beta) \cdot z^2(t_i) \tag{16}$$

em que $\beta = 0,95$ e $d_m(t_i)$ é medido de acordo com (4). Essa é uma metodologia típica empregada quando não há informação disponível sobre o ruído de medição [2].

Por fim, quanto às condições iniciais do sistema, a convergência rápida é garantida quando a variância inicial do erro do sistema $P(0)$ é maior que a variância do ruído de estado Q. Definimos $P(0) = 10^{-1}$. Nessas condições, a estimativa inicial do estado pode ser definida livremente com qualquer valor (escolhemos $m(0) = 0$).

4.2 Projeto do limiar adaptativo

O objetivo do bloco de limiar adaptativo da Figura 1 é adaptar a sensibilidade do algoritmo ao gradiente de atraso conforme as condições da rede. Mostramos que o limiar $\gamma(t_i)$, usado pelo *detector de sobreuso*, precisa ser adaptativo; caso contrário, podem ocorrer dois problemas: 1) a ação de controle baseada em atraso pode não ter nenhum efeito quando o tamanho da fila do gargalo ao longo do caminho não é suficientemente grande e 2) o fluxo GCC pode sofrer starvation por causa de um fluxo TCP concorrente baseado em perdas. Para isso, propomos o seguinte limiar adaptativo:

$$\gamma(t_i) = \gamma(t_{i-1}) + \Delta T \cdot k_\gamma(t_i)(|m(t_i)| - \gamma(t_{i-1})) \tag{17}$$

em que $\Delta T = t_i - t_{i-1}$ e t_i é o instante em que o i-ésimo quadro de vídeo é recebido. O ganho $k_\gamma(t_i)$ é definido da seguinte forma:

$$k_\gamma(t_i) = \begin{cases} k_d & |m(t_i)| < \gamma(t_{i-1}) \\ k_u & \text{caso contrário} \end{cases} \tag{18}$$

em que k_u e k_d determinam, respectivamente, a velocidade com que o limiar é aumentado ou diminuído. Com (17), o controlador baseado em atraso no receptor compara o gradiente de atraso unidirecional $m(t_i)$ a um limiar $\gamma(t_i)$ que é uma versão filtrada por passa-baixas de $|m(t_i)|$. A lógica é a seguinte: quando $m(t_i)$ ultrapassa $\gamma(t_i)$, o controlador baseado em atraso reduz $m(t_i)$ e o limiar $\gamma(t_i)$ segue $m(t_i)$ com uma constante de tempo mais lenta, de modo que $m(t_i)$ permaneça abaixo de $\gamma(t_i)$ e conduza o algoritmo ao estado de redução. Isso continua até que $m(t_i)$ ultrapasse novamente $\gamma(t_i)$ e o algoritmo baseado em atraso volte a reduzir $m(t_i)$. Essa adaptação evita a starvation do fluxo GCC quando um fluxo TCP entra no gargalo. Vale notar que, quando $\gamma(t_i) > m(t_i)$, $\gamma(t_i)$ segue $m(t_i)$ com uma constante de tempo menor, de modo que menos eventos de redução são necessários para tornar $m(t_i) < \gamma(t_i)$.

Escolha dos parâmetros do limiar. Uma característica fundamental do projeto do limiar adaptativo reside no ajuste dos parâmetros k_u e k_d, que definem a constante de tempo com que o limiar $\gamma(t_i)$ se adapta ao gradiente de atraso. Em outras palavras, os parâmetros k_u e k_d determinam a sensibilidade do algoritmo ao atraso unidirecional estimado $m(t)$. Para ajustar

esses parâmetros, definimos um problema de otimização empregando a função objetivo proposta em [28]:

$$U(\boldsymbol{x}) = \sum_{i=1}^{N} U^{\alpha}(x_i) \tag{19}$$

em que $U^{\alpha}(x_i)$ é a função objetivo medida para o i-ésimo fluxo. A utilidade global é obtida como a soma de $U^{\alpha}(x_i)$ para cada um dos N fluxos concorrentes. x_i é o throughput médio do i-ésimo fluxo e $U^{\alpha}(x)$ é uma função côncava dada por:

$$U^{\alpha}(x_i) = \frac{x_i^{1-\alpha}}{1-\alpha} \tag{20}$$

Para qualquer valor de $\alpha > 0$, esse problema de otimização é Pareto-eficiente [28]. A maximização de U implica uma alocação justa do throughput entre os fluxos concorrentes. No caso da videoconferência, precisamos estender (19) para considerar o impacto do atraso de fila sobre o desempenho do sistema [31]:

$$U(\boldsymbol{x}, \boldsymbol{y}) = \sum_{i=1}^{N} \big(U^{\alpha}(x_i) - \delta \cdot U^{\beta}(y_i) \big) \tag{21}$$

A mesma lógica usada para $U^{\alpha}(x_i)$ aplica-se a $U^{\beta}(y_i)$, em que y_i se refere ao atraso de fila medido para o i-ésimo fluxo. Os parâmetros α e β expressam o compromisso entre justiça e eficiência para throughput e atraso, enquanto δ expressa a importância relativa do atraso em relação ao throughput. Seguindo a lógica de [31], usamos $\alpha = 2, \beta = 1$, isto é, damos mais ênfase à justiça de throughput, e $\delta = 0,15$, que garante um bom equilíbrio entre $U^{\alpha}(x_i)$ e $U^{\beta}(y_i)$.

Definimos três cenários diferentes nos quais medimos a função objetivo (21). Trata-se de três cenários de avaliação descritos na Seção 6: 1) um único fluxo GCC sobre um gargalo com capacidade de enlace variável (Seção 6.1); 2) múltiplos fluxos GCC concorrentes sobre um gargalo com capacidade de enlace constante (Seção 6.2); 3) um fluxo GCC contra um fluxo TCP (de forma similar ao caso da Seção 6.3). A função objetivo foi calculada para cada valor de k_u e k_d no intervalo $[10^{-5}, 0, 1] \times [10^{-5}, 0, 1]$ dividido em 200×200 intervalos em escala logarítmica. O mapa de contorno da soma das funções objetivo normalizadas U obtidas em cada cenário para cada par (k_u, k_d) é mostrado na Figura 8. U aumenta quando $k_u > k_d$, isto é, quando a constante de tempo com que $\gamma(t_i)$ é aumentado é menor que a constante de tempo com que ele é diminuído. Porém, quando $k_u \gg k_d$ (no canto inferior direito da Figura 8), o limiar diminui devagar demais, reduzindo a sensibilidade do algoritmo à inflação de atraso; isso induz grandes atrasos de fila e, consequentemente, U diminui. Por outro lado, quando $k_u < k_d$, o algoritmo torna-se muito sensível ao gradiente de atraso, o que leva à degradação do throughput na presença de fluxos TCP concorrentes. O valor ótimo é obtido para $(k_u, k_d) = (0,01, 0,00018)$. Isso garante um bom equilíbrio entre alto throughput, redução de atraso e justiça intra e interprotocolo.

Figura 8: Gráfico de contorno da função objetivo U em função de k_u e k_d.
(imagem omitida nesta tradução; consulte o artigo original)

Influência do tamanho da fila do gargalo sobre o gradiente de atraso. Neste parágrafo mostramos por que o limiar precisa ser adaptativo para evitar a inibição do algoritmo baseado em atraso quando o tamanho da fila do gargalo não é suficientemente grande. Para isso, consideramos o caso de um único fluxo acessando um gargalo. Mostramos que pode surgir um fenômeno indesejável se o limiar for definido estaticamente como um valor $\overline{\gamma}$: em particular, se $\overline{\gamma}$ for maior que o gradiente de atraso máximo mensurável m_M, o algoritmo baseado em atraso não tem efeito algum. A Figura 9(a) mostra um experimento real (ver Seção 5 para detalhes do testbed) em que um único fluxo GCC entra em um enlace gargalo de 1 Mbps com uma fila drop-tail cujo tempo máximo de fila é $T_q = 150$ ms. A Figura 9 compara a taxa GCC, o RTT e a dinâmica da fração de perdas no caso de um limiar estático (Figura 9(a)) e no caso do limiar adaptativo (Figura 9(b)). Com um limiar estático $\overline{\gamma}$, o controlador baseado em atraso torna-se ineficaz e não consegue reagir à inflação de atraso. Isso se deve ao fato de que o valor máximo do gradiente de atraso m_M, que depende de T_q, é menor que $\overline{\gamma}$, isto é, o valor de $\overline{\gamma}$ é grande demais para $T_q = 150$ ms. A Figura 9(b) mostra que, com o limiar adaptativo, $\gamma(t)$ segue $m(t)$ com uma constante de tempo mais lenta e, quando $m(t)$ ultrapassa $\gamma(t)$, o algoritmo baseado em atraso consegue reduzir a taxa de envio. Além disso, como $k_u > k_d$, o limiar aumenta mais rápido do que diminui. A Figura 9(b) também mostra que o controlador consegue evitar perdas de pacotes, isto é, $f_l(t) = 0$ durante toda a duração do experimento. Para mostrar mais claramente os benefícios do limiar adaptativo, a Figura 10 compara a função de distribuição acumulada (CDF) do RTT medido nos dois experimentos apresentados na Figura 9. O valor mediano do RTT é muito próximo do atraso de propagação $\mathrm{RTT}_{\min} = 50$ ms definido pelo testbed em ambos os casos, ao passo que, graças ao limiar adaptativo, o percentil 95 do RTT é reduzido de 187 ms para 130 ms.

Figura 9: Efeito do limiar adaptativo no caso de um único fluxo GCC sobre um enlace de 1 Mbps com tamanho de fila $T_q = 150$ ms. (a) Limiar estático; (b) limiar adaptativo.
(imagem omitida nesta tradução; consulte o artigo original)

Figura 10: Comparação de RTT no caso de um único fluxo GCC (CDF do RTT: percentil 95 de 187 ms com limiar estático e de 130 ms com limiar adaptativo).
(imagem omitida nesta tradução; consulte o artigo original)

Efeito de um fluxo TCP concorrente sobre o gradiente de atraso. Neste parágrafo mostramos que o limiar precisa ser adaptativo para evitar a starvation de um fluxo GCC na presença de um fluxo concorrente baseado em perdas. Para isso, consideramos um único fluxo GCC com um fluxo TCP concorrente de longa duração. Mostramos que um limiar estático pode levar à starvation do fluxo GCC. Nesse cenário, o gradiente de atraso unidirecional pode ser expresso como a soma de dois componentes:

$$m(t) = m_{\mathrm{GCC}}(t) + m_{\mathrm{TCP}}(t) \tag{22}$$

em que $m_{\mathrm{GCC}}(t)$ e $m_{\mathrm{TCP}}(t)$ são os gradientes de atraso do fluxo GCC e do fluxo TCP, respectivamente. Pode-se mostrar que o gradiente de atraso máximo $m_{\mathrm{TCP},M}$ devido a um fluxo TCP pode ser muito maior que o de um fluxo GCC [6]. Como, nesse caso, $m(t)$ contém o componente $m_{\mathrm{TCP}}(t)$, se $m_{\mathrm{TCP}}(t) \gg m_{\mathrm{GCC}}(t)$, o fluxo GCC reduzirá a taxa de envio não por causa do atraso autoinfligido, mas por causa do fluxo TCP.

A Figura 11 mostra um experimento real, conduzido com o testbed descrito na Seção 5, em que um fluxo GCC compete com um fluxo TCP Cubic sobre um enlace gargalo de 1 Mbps com fila drop-tail cujo tempo máximo de fila é $T_q = 350$ ms. A Figura 11(a) mostra que, quando se usa um limiar estático, o fluxo GCC sofre starvation. Em particular, quando o fluxo TCP é iniciado, $m(t)$ começa a oscilar acima do limiar γ, principalmente devido ao gradiente de atraso do fluxo TCP $m_{\text{TCP}}(t)$, o que dispara um grande número de sinais de sobreuso. Consequentemente, a FSM do controlador de taxa remoto entra no modo de redução, que diminui o valor de A_r de acordo com (3). Por outro lado, a Figura 11(b) mostra que o limiar adaptativo consegue resolver o problema de starvation e permite que o fluxo GCC compartilhe o gargalo de forma justa com o fluxo TCP concorrente. Em particular, depois que o TCP é iniciado, $\gamma(t)$ segue $m(t)$ com uma constante de tempo menor, o que evita a geração de um grande número de sinais de sobreuso consecutivos e impede a starvation do fluxo GCC.

Figura 11: Um fluxo GCC contra um fluxo TCP. Parâmetros do gargalo: tamanho da fila $T_q = 350$ ms, capacidade do enlace $C = 1000$ kbps. (a) Limiar estático; (b) limiar adaptativo.
(imagem omitida nesta tradução; consulte o artigo original)

5 TESTBED

A Figura 12 mostra o testbed experimental empregado para emular um cenário de WAN. Ele consiste em quatro máquinas Linux com kernel 3.16.0. Dois nós, cada um executando várias sessões de navegadores Chromium[4] e uma aplicação para gerar ou receber fluxos TCP de longa duração, são conectados por um cabo Ethernet. Com essa configuração, é possível analisar o comportamento dos fluxos GCC ao competirem com fluxos TCP. Outro nó executa um servidor Web que cuida da sinalização necessária para estabelecer as chamadas de vídeo entre os navegadores. O último nó é o controlador do testbed, que orquestra os experimentos por meio de comandos ssh. O controlador do testbed realiza as seguintes tarefas: 1) coloca as chamadas WebRTC, iniciando os fluxos GCC; 2) define a capacidade do enlace C e o tamanho da fila do gargalo T_q no Nó 1; 3) define o atraso de propagação RTT_{min} no Nó 2[5] 4) inicia os fluxos TCP quando necessário. O tamanho da fila q_M no Nó 1 foi definido considerando o tempo máximo necessário para drenar o buffer, T_q, isto é, $q_M = T_q \cdot C$. O atraso de propagação de ida e volta $\text{RTT}_{\text{min}} = 50$ ms, considerado como a soma dos atrasos de propagação nos caminhos direto e reverso (25 ms cada), foi definido no Nó 2 por meio do módulo NetEm do Linux, enquanto o tamanho do buffer do gargalo foi definido como $T_q = 300$ ms, seguindo os critérios de avaliação fornecidos em [25]. Usamos um Token Bucket Filter (TBF) para definir a capacidade C do enlace de ingresso do Nó 1. Desativamos os parâmetros de otimização da NIC, isto é, TCP segmentation offload, jumbo frames e generic segmentation offload, pois poderiam interferir nos experimentos.

Figura 12: Testbed experimental (Nó 1 e Nó 2 com fontes/sinks TCP e traffic shapers, controlador do testbed, servidor de sinalização; fluxos GCC sobre RTP/UDP).
(imagem omitida nesta tradução; consulte o artigo original)

[4]https://chromium.googlesource.com/chromium/src.git
[5]Recomenda-se separar o controle de taxa da emulação de atraso.

Configurações de vídeo e TCP. As fontes TCP empregam o controle de congestionamento CUBIC, que é o padrão no kernel do Linux. São registrados a janela de congestionamento, o limiar de slow-start, o RTT e o número de sequência. Um servidor Web[6] fornece a página HTML que cuida da sinalização entre os pares usando a API JavaScript do WebRTC. A mesma sequência de vídeo é usada para garantir a reprodutibilidade dos experimentos. Para isso, usamos o módulo de kernel Linux v4l2loopback[7] para criar um dispositivo de webcam virtual que repete ciclicamente a sequência de teste YUV "Four People"[8]. O Chromium codifica a fonte de vídeo bruta com o codificador de vídeo VP8[9]. Medimos que, sem nenhuma limitação de largura de banda, o VP8 limita a taxa de envio $A_s(t)$ a um valor máximo de 2 Mbps.

Métricas. Para avaliar quantitativamente o desempenho do GCC, consideramos métricas de QoS como razão de perda de pacotes, taxa de bits média e atraso, que são sabidamente bem correlacionadas com métricas de QoE por meio, por exemplo, da hipótese IQX [10]. Seguir essa abordagem tem o mérito de concentrar a discussão em métricas que não são sensíveis a aspectos específicos da aplicação, como o codificador de vídeo empregado. Além disso, separar a avaliação de métricas de QoE das de QoS também segue as diretrizes definidas recentemente no grupo de trabalho RTP Media Congestion Avoidance Techniques (RMCAT) do IETF. Em particular, consideramos:

- Utilização do canal $U = R_r/C$, em que C é a capacidade conhecida do enlace e R_r é a taxa média recebida;
- Razão de perda l = (bytes perdidos)/(bytes enviados);
- Percentis 50 e 95 do atraso de fila, medido como $\text{RTT}(t) - \text{RTT}_{\text{min}}$ sobre todas as amostras de RTT reportadas nos feedbacks RTCP durante os experimentos;
- Índice de Justiça de Jain: $\text{JFI}(t) = \frac{\left(\sum_{i=1}^{N} x_i(t)\right)^2}{N\sum_{i=1}^{N} x_i(t)^2}$, em que $x_i(t)$ é o throughput instantâneo medido do i-ésimo fluxo e N é o número total de fluxos concorrentes.

6 AVALIAÇÃO EXPERIMENTAL

Esta seção apresenta os resultados experimentais obtidos com o testbed descrito na Seção 5. A análise baseia-se nos critérios de avaliação em discussão no grupo de trabalho RMCAT do IETF [25]. Vale mencionar que uma avaliação experimental muito mais ampla foi realizada, mas, por limitações de espaço, decidimos incluir apenas resultados experimentais dos cenários definidos em [25]. Argumentamos que essa escolha facilita comparações experimentais com outros algoritmos propostos no RMCAT. Em particular, o objetivo é verificar se o GCC satisfaz os requisitos definidos em [24], isto é, baixo enfileiramento na ausência de tráfego heterogêneo concorrente e uma parcela razoável da largura de banda ao competir com outros fluxos homogêneos ou heterogêneos. Para cada cenário foram executados dez experimentos. Para cada um deles é fornecida uma tabela que contém o valor médio das métricas medidas nas dez execuções.

[6]https://apprtc.appspot.com/
[7]https://github.com/umlaeute/v4l2loopback
[8]https://people.xiph.org/~thdavies/x264_streams/FourPeople_1280x720_30/
[9]http://www.webmproject.org/

6.1 Um único GCC com capacidade de enlace variável

Este cenário investiga com que rapidez a taxa de bits do vídeo controlada pelo GCC reage a mudanças súbitas na capacidade do enlace. Para isso, variamos a capacidade do enlace a cada intervalo de 50 segundos. Consideramos quatro intervalos de tempo nos quais a capacidade é definida, respectivamente, como 1 Mbps, 2,5 Mbps, 0,5 Mbps e 1 Mbps, de modo que a chamada de vídeo dura 200 s. A Tabela 1 mostra o valor médio das métricas medidas ao longo dos dez experimentos. A Figura 13 mostra a dinâmica da taxa GCC, do RTT e da fração de perdas. Esse experimento mostra que o GCC acompanha a capacidade do enlace variável no tempo. É importante observar que, no intervalo de tempo em que a capacidade é definida como 2,5 Mbps, o fluxo GCC não consegue igualar a capacidade do enlace, pois o codificador de vídeo não produz mais que 2 Mbps. Além disso, o GCC consegue conter o atraso de fila, pois o RTT é mantido próximo ao atraso de propagação de ida e volta $\mathrm{RTT}_{\mathrm{min}} = 50$ ms durante toda a chamada de vídeo. Medimos um valor médio do percentil 50 do atraso de fila igual a 20 ms, enquanto o percentil 95 é de 195 ms. Notamos que, para reagir à queda da capacidade do enlace, o algoritmo precisa reduzir a taxa de envio para manter baixos o atraso de fila e a razão de perda. Quando a capacidade do enlace aumenta, o GCC leva cerca de 25 segundos para alcançar o novo valor de capacidade. Isso resulta em uma utilização média do canal igual a 84%.

Figura 13: Dinâmica de taxa, RTT e fração de perdas no caso de um único GCC com capacidade de enlace variável.
(imagem omitida nesta tradução; consulte o artigo original)

Métrica	Valor
Utilização do canal	84%
Percentil de enfileiramento (50º – 95º)	20 ms – 195 ms
Razão de perda	0,02%

Tabela 1: Valor médio das métricas medidas no caso de um único GCC com capacidade de enlace variável.

6.2 Justiça intraprotocolo

O objetivo deste cenário é investigar a justiça intraprotocolo do GCC. Para isso, consideramos três fluxos GCC concorrentes sobre um enlace de 3 Mbps. Cada fluxo é iniciado 30 segundos depois do anterior. O experimento dura 200 segundos. A Tabela 2 mostra o valor médio das métricas medidas ao longo de dez experimentos quando os três fluxos estão ativos. A Figura 14 mostra a dinâmica de um experimento. Esse experimento mostra que o GCC não é afetado pelo "efeito latecomer" [7]; os três fluxos GCC compartilham o enlace de forma justa e o Índice de Justiça de Jain medido se aproxima de 0,93. A convergência pode ser mostrada intuitivamente considerando que o gradiente de atraso estimado equivalente medido por cada fluxo é dado por:

$$m(t) = m_{\mathrm{GCC1}}(t) + m_{\mathrm{GCC2}}(t) + m_{\mathrm{GCC3}}(t) \tag{23}$$

de modo que um aumento do gradiente de atraso induzido por um dos fluxos dispara um "sinal de sobreuso" em todos os fluxos concorrentes. Como a taxa de bits é reduzida de acordo com (3), um fluxo com taxa de bits maior sofrerá uma redução de taxa maior, levando, por fim, à convergência. Nenhum pacote é perdido durante toda a duração do experimento. Quanto ao atraso de fila, neste experimento medimos um valor médio do percentil 50 igual a 10 ms ($\mathrm{RTT}_{\mathrm{min}} = 50$ ms), enquanto o percentil 95 medido é igual a 61 ms.

Figura 14: Dinâmica de taxa, fração de perdas e RTT no caso de três fluxos GCC concorrentes sobre um enlace de 3 Mbps.
(imagem omitida nesta tradução; consulte o artigo original)

Métrica	Valor
Utilização cumulativa	86%
Percentil de enfileiramento (50º – 95º)	10 ms – 61 ms
Razão de perda cumulativa	0%
Índice de Justiça de Jain	0,93

Tabela 2: Valor médio das métricas medidas no caso de três fluxos GCC concorrentes sobre um enlace de 3 Mbps.

Para completar a investigação sobre a justiça intraprotocolo, consideramos um cenário com dois fluxos no caso de um enlace com capacidade variável. A capacidade do enlace é variada a cada intervalo de 25 segundos. Consideramos 5 intervalos nos quais a capacidade é definida, respectivamente, como 4 Mbps, 2 Mbps, 4 Mbps, 1 Mbps e 2 Mbps, de modo que a chamada de vídeo dura 125 segundos. A Tabela 3 mostra o valor médio das métricas medidas ao longo dos dez experimentos. A Figura 15 mostra a dinâmica das variáveis em um experimento. O Índice de Justiça de Jain é igual a 0,87, o que confirma que o GCC proporciona justiça intraprotocolo.

Figura 15: Dinâmica de taxa, fração de perdas e RTT no caso de dois fluxos GCC sobre um enlace com capacidade variável.
(imagem omitida nesta tradução; consulte o artigo original)

Métrica	Valor
Utilização cumulativa	85%
Percentil de enfileiramento (50º – 95º)	15 ms – 103 ms
Razão de perda cumulativa	1%
Índice de Justiça de Jain	0,87

Tabela 3: Valor médio das métricas medidas no caso de dois fluxos GCC sobre um enlace com capacidade variável.

6.3 Um fluxo GCC com fluxos TCP concorrentes de longa duração

Este cenário visa testar a justiça interprotocolo. Consideramos um fluxo GCC contra 99 fluxos TCP de longa duração sobre um enlace gargalo com capacidade constante de 100 Mbps. A chamada de vídeo dura 400 segundos e os fluxos TCP ficam ativos por 200 segundos, no intervalo de tempo [100, 300] segundos. A Tabela 4 (no original: "Table 2") resume as métricas obtidas nos dez experimentos, medidas enquanto os fluxos TCP estão ativos. A Figura 16 mostra a dinâmica obtida em um dos experimentos realizados. A figura mostra que, assim que os fluxos TCP entram no gargalo, a taxa de envio do GCC é reduzida para próximo da parcela justa (fair share) de 1 Mbps e oscila em torno desse valor enquanto os fluxos TCP estão ativos. Isso é possibilitado pelo limiar adaptativo (ver Seção 4.2) que, ao reduzir a sensibilidade do controlador baseado em atraso, faz com que o fluxo GCC seja controlado tanto pelo algoritmo baseado em perdas quanto pelo baseado em atraso na presença de tráfego concorrente baseado

em perdas. De fato, o valor da fração de perdas medido para o fluxo GCC é maior que zero quando os fluxos TCP estão ativos, o que confirma que o algoritmo também opera em modo baseado em perdas. Como esperado, o atraso de fila não pode ser contido na presença de tráfego TCP, que tende a encher o buffer do gargalo, atrapalhando a interação em tempo real.

Figura 16: O caso de um fluxo GCC compartilhando um enlace de 100 Mbps com 99 fluxos TCP.
(imagem omitida nesta tradução; consulte o artigo original)

Métrica	Valor
Throughput do GCC	1,3 Mbps
Percentil de enfileiramento (50º – 95º)	250 ms – 295 ms
Razão de perda	1,2%
Índice de Justiça de Jain	0,81

Tabela 4: Valor médio das métricas medidas no caso de um fluxo GCC compartilhando um enlace de 100 Mbps com 99 fluxos TCP.

6.4 Fluxos GCC na presença de fluxos TCP de curta duração

O objetivo deste cenário é investigar como os fluxos GCC se comportam na presença de tráfego TCP de curta duração. Para isso, consideramos dois fluxos GCC concorrentes sobre um enlace de 2 Mbps; 5 segundos após o início das chamadas de vídeo, iniciamos um fluxo TCP por 3 segundos a cada 12 segundos. Esse cenário é interessante porque a interação entre os fluxos difere do caso de um fluxo TCP de longa duração: os fluxos TCP de curta duração provocam várias rajadas de tráfego, típicas no caso de transferência de arquivos pequenos (por exemplo, download de uma página Web). A Tabela 5 resume as métricas obtidas para os dois fluxos GCC, enquanto a Figura 17 mostra a dinâmica de um experimento: durante o intervalo de tempo em que o fluxo TCP está ativo (representado em cinza), o RTT aumenta devido aos fluxos TCP, que inflam o buffer do gargalo. Quando o tráfego TCP não está presente, as taxas de envio dos fluxos GCC aumentam até 1 Mbps, que é a parcela justa ideal de largura de banda para dois fluxos concorrentes. Quando o TCP é iniciado, a taxa de envio dos fluxos é reduzida devido à inflação de atraso induzida. Por isso, a utilização global do canal medida para os dois fluxos GCC é igual a 72%.

Figura 17: Dinâmica de taxa, fração de perdas e RTT de dois fluxos GCC na presença de fluxos TCP de curta duração.
(imagem omitida nesta tradução; consulte o artigo original)

Métrica	Valor
Utilização dos fluxos GCC	72%
Percentil de enfileiramento (50º – 95º)	109 ms – 285 ms
Razão de perda	0,7%
Índice de Justiça de Jain	0,78

Tabela 5: Valor médio das métricas medidas no caso de dois fluxos GCC na presença de fluxos TCP de curta duração.

6.5 O efeito do tráfego reverso

Este cenário avalia o efeito do tráfego reverso sobre as métricas. Para isso, uma chamada de vídeo é estabelecida no caminho direto, cuja capacidade é de 1 Mbps, por 100 segundos. Durante a chamada, em um enlace de caminho reverso de 1 Mbps, um fluxo TCP é iniciado por 40 segundos. A Tabela 6 resume o valor médio das métricas obtidas para os fluxos GCC considerando todas as execuções. A Figura 18 mostra a dinâmica de um resultado experimental: representamos em cinza o intervalo de tempo durante o qual o fluxo TCP está ativo no caminho reverso. Esse teste mostra claramente que a taxa GCC e a fração de perdas não são afetadas pelo tráfego reverso. Isso se deve ao fato de que o gradiente de atraso estimado no caminho direto não é afetado pela presença de tráfego reverso, embora a inflação do RTT confirme a presença do fluxo TCP. A utilização média do canal medida no caminho direto é de 90%. O atraso de fila mediano medido no caminho direto é de 10 ms e o percentil 95 é de 159 ms.

Figura 18: Dinâmica de taxa, fração de perdas, RTT e gradiente de atraso para o fluxo GCC no caminho direto.
(imagem omitida nesta tradução; consulte o artigo original)

Métrica	Valor
Utilização do canal	90%
Percentil de enfileiramento (50º – 95º)	10 ms – 159 ms
Razão de perda	0%

Tabela 6: Valor médio das métricas medidas para o fluxo GCC no caminho direto.

7 CONCLUSÕES E TRABALHOS FUTUROS

Neste artigo apresentamos um algoritmo de controle de congestionamento para fluxos de tempo real que é usado atualmente no Google Hangouts. Realizamos uma avaliação experimental baseada nas diretrizes em definição no grupo de trabalho RMCAT do IETF. Os resultados obtidos com o navegador Google Chromium mostram que o algoritmo: 1) adapta a taxa de envio para acompanhar a capacidade do enlace; 2) proporciona justiça intraprotocolo e interprotocolo com fluxos TCP de longa e curta duração; e 3) não é afetado pelo tráfego reverso. Como trabalho futuro, planejamos uma campanha de medições em larga escala envolvendo um grande conjunto de usuários do Chrome.

8 AGRADECIMENTOS

Este trabalho foi parcialmente financiado pelo Ministério italiano da Educação, Universidades e Pesquisa (MIUR) por meio do projeto MAIVISTO (PAC02L1 00061). Quaisquer opiniões, constatações, conclusões ou recomendações expressas neste material são dos autores e não refletem necessariamente as opiniões das agências financiadoras.

9 REFERÊNCIAS (NO IDIOMA ORIGINAL)

[1] A. Bergkvist, D. C. Burnett, C. Jennings, and A. Narayanan. Webrtc 1.0: Real-time communication between browsers. W3C Working Draft, Feb. 2015.
[2] R. Bos, X. Bombois, and P. M. Van den Hof. Designing a Kalman filter when no noise covariance information is available. In Proc. of the 16th IFAC World Congress, volume 16, pages 212–212, Jul. 2005.
[3] L. S. Brakmo and L. L. Peterson. TCP Vegas: End to end congestion avoidance on a global Internet. IEEE Journal on Selected Areas in Communicationss, 13(8):1465–1480, Oct. 1995.

[4] B. Briscoe, A. Brunstrom, A. Petlund, D. Hayes, D. Ros, J. Tsang, S. Gjessing, G. Fairhurst, C. Griwodz, and M. Welzl. Reducing Internet Latency: A Survey of Techniques and their Merits. IEEE Comm. Surveys & Tutorials, in press.
[5] E. Brosh, S. Baset, V. Misra, D. Rubenstein, and H. Schulzrinne. The Delay--Friendliness of TCP for Real-Time Traffic. IEEE/ACM Transactions on Networking, 18(5):1478–1491, Oct. 2010.
[6] G. Carlucci, L. De Cicco, and S. Mascolo. Modelling and Control for Web Real-Time Communication. In 53rd IEEE Conference on Decision and Control, Los Angeles, CA, USA, Dec. 2014.
[7] G. Carofiglio, L. Muscariello, D. Rossi, and S. Valenti. The Quest for LEDBAT Fairness. In IEEE Global Telecommunications Conference, pages 1–6, Dec. 2010.
[8] L. De Cicco and S. Mascolo. A mathematical model of the Skype VoIP congestion control algorithm. IEEE Transactions on Automatic Control, 55(3):790–795, Mar. 2010.
[9] J. Durbin and S. J. Koopman. Time series analysis by state space methods. Oxford University Press, 2012.
[10] M. Fiedler, T. Hossfeld, and P. Tran-Gia. A generic quantitative relationship between quality of experience and quality of service. IEEE Network, 24(2):36–41, 2010.
[11] G. F. Franklin, J. D. Powell, and M. L. Workman. Digital control of dynamic systems, volume 3. Addison-wesley Menlo Park, 1998.
[12] L. A. Grieco and S. Mascolo. Performance evaluation and comparison of Westwood+, New Reno, and Vegas TCP congestion control. ACM SIGCOMM CCR, 34(2):25–38, Apr. 2004.
[13] A. Gurtov and S. Floyd. Modeling wireless links for transport protocols. ACM SIGCOMM CCR, 34(2):85–96, Apr. 2004.
[14] D. A. Hayes and G. Armitage. Revisiting TCP Congestion Control Using Delay Gradients. In Proceedings of the 10th International IFIP TC 6 Conference on Networking - Volume Part II, pages 328–341, Jul. 2011.
[15] S. Holmer, H. Lundin, G. Carlucci, L. De Cicco, and S. Mascolo. Google Congestion Control Algorithm for Real-Time Communication on the World Wide Web. IETF Draft, June 2015.
[16] R. Jain. A delay-based approach for congestion avoidance in interconnected heterogeneous computer networks. ACM SIGCOMM CCR, 19(5):56–71, Oct. 1989.
[17] I. Johansson. Self-clocked Rate Adaptation for Conversational Video in LTE. In Proceedings of the 2014 ACM SIGCOMM Workshop on Capacity Sharing Workshop, pages 51–56, Chicago, Illinois, USA, Aug. 2014.
[18] A. Kuzmanovic and E. Knightly. TCP-LP: low-priority service via end--point congestion control. Networking, IEEE/ACM Transactions on, 14(4):739–752, Aug 2006.
[19] R. Mittal, V. T. Lam, N. Dukkipati, E. Blem, H. Wassel, M. Ghobadi, A. Vahdat, Y. Wang, D. Wetherall, and D. Zats. TIMELY: RTT-based Congestion Control for the Datacenter. ACM SIGCOMM CCR, 45(5):537–550, Oct. 2015.
[20] M. Nagy, V. Singh, J. Ott, and L. Eggert. Congestion Control Using FEC for Conversational Multimedia Communication. In Proceedings of the 5th ACM Multimedia Systems Conference, pages 191–202, Singapore, Mar. 2014.
[21] P. Ni, R. Eg, A. Eichhorn, C. Griwodz, and P. Halvorsen. Flicker effects in adaptive video streaming to handheld devices. In Proc. of ACM International Conference on Multimedia, pages 463–472, Nov. 2011.
[22] K. Nichols and V. Jacobson. Controlling queue delay. Queue, ACM, 10(5):20:20–20:34, May 2012.
[23] R. S. Prasad, M. Jain, and C. Dovrolis. On the effectiveness of delay-based congestion avoidance. In Proc. of Workshop on Protocols for Fast Long--Distance Networks, volume 4, Feb. 2004.
[24] J. Randell and Z. Sarker. Congestion control requirements for RMCAT. Draft IETF, Dec. 2014.
[25] Z. Sarker, V. Singh, X. Zhu, and R. M. Test Cases for Evaluating RMCAT Proposals. IETF Draft, Aug. 2015.
[26] S. Shalunov, G. Hazel, J. Iyengar, and M. Kuehlewind. Low extra delay background transport (LEDBAT). RFC 6817, Dec. 2012.
[27] D. Sisalem and A. Wolisz. Lda+: a tcp-friendly adaptation scheme for multimedia communication. In IEEE International Conference on Multimedia and Expo, volume 3, pages 1619–1622 vol.3, Aug. 2000.
[28] R. Srikant. The mathematics of Internet congestion control. Springer Science & Business Media, 2012.
[29] B. Wang, J. Kurose, P. Shenoy, and D. Towsley. Multimedia streaming via tcp: An analytic performance study. ACM Trans. Multimedia Comput. Commun. Appl., 4(2):16:1–16:22, May 2008.
[30] D. X. Wei, C. Jin, S. H. Low, and S. Hegde. FAST TCP: motivation, architecture, algorithms, performance. IEEE/ACM Transactions on Networking, 14(6):1246–1259, 2006.
[31] K. Winstein and H. Balakrishnan. TCP Ex Machina: Computer-generated Congestion Control. ACM SIGCOMM CCR, 43(4):123–134, Oct. 2013.
[32] K. Winstein, A. Sivaraman, and H. Balakrishnan. Stochastic Forecasts Achieve High Throughput and Low Delay over Cellular Networks. In Proc. of USENIX NSDI, Apr. 2013.
[33] Y. Xu, C. Yu, J. Li, and Y. Liu. Video Telephony for End-Consumers: Measurement Study of Google+, iChat, and Skype. IEEE/ACM Transactions on Networking, 22(3):826–839, Jun. 2014.
[34] Y. Zaki, T. Pötsch, J. Chen, L. Subramanian, and C. Görg. Adaptive congestion control for unpredictable cellular networks. ACM SIGCOMM CCR, 45(5):509–522, Oct. 2015.
[35] X. Zhu, R. Pan, S. Mena, P. Jones, J. Fu, S. D'Aronco, and C. Ganzhorn. Nada: A unified congestion control scheme for real-time media. IETF Draft, Mar. 2015.